Maven and Docker Integration Using dockerfile-maven-plugin

Complete step-by-step practical: open, edit, save, close, build, run, and verify

Objective: Create a Java Maven project, add Docker integration using the Spotify dockerfile-maven-plugin, build a Docker image through Maven, and run the application inside a Docker container.

Important note: The Spotify dockerfile-maven-plugin repository is archived/read-only, but it is still useful for classroom learning and basic Maven-Docker integration practice.

Prerequisites

- Java JDK 17 installed.

- Apache Maven installed.

- Docker Desktop or Docker Engine installed and running.

- Any code editor such as VS Code, IntelliJ IDEA, Eclipse, or Notepad++.

- Internet connection for Maven dependency download.

Step 1: Open Terminal or Command Prompt

Open Terminal in Linux/macOS or Command Prompt/PowerShell in Windows. Check Maven, Java, and Docker installation using these commands:

```
java -version
mvn -version
docker --version
```

If all commands show version details, continue to the next step.

Step 2: Create a New Maven Project

Run the following command to create a Maven quickstart Java project:

```
mvn archetype:generate -DgroupId=com.example -DartifactId=maven-docker-demo
    -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

This command creates a folder named maven-docker-demo.

Step 3: Open the Project Folder

Move inside the newly created project folder:

```
cd maven-docker-demo
```

Now open this folder in your code editor. Example command for VS Code:
code .

Step 4: Understand the Project Structure

After opening the project, you should see this structure:

```
maven-docker-demo/
 +-- pom.xml
 +-- src/
     +-- main/
     |   +-- java/com/example/App.java
     +-- test/
         +-- java/com/example/AppTest.java
```

Step 5: Open pom.xml

Open the pom.xml file from the root folder of the project. This file controls Maven build settings, dependencies, and plugins.

Replace the complete content of pom.xml with the following code:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>maven-docker-demo</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>jar</packaging>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.11.0</version>
                <configuration>
                    <source>17</source>
                    <target>17</target>
                </configuration>
            </plugin>

            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-jar-plugin</artifactId>
                <version>3.3.0</version>
                <configuration>
                    <archive>
                        <manifest>
                            <mainClass>com.example.App</mainClass>
                        </manifest>
                    </archive>
                </configuration>
            </plugin>

            <plugin>
                <groupId>com.spotify</groupId>
                <artifactId>dockerfile-maven-plugin</artifactId>
                <version>1.4.13</version>
                <configuration>
                    <repository>maven-docker-demo</repository>
                    <tag>latest</tag>
                    <buildArgs>
                        <JAR_FILE>target/${project.build.finalName}.jar</JAR_FILE>
                    </buildArgs>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

Save and close pom.xml.

Step 6: Open App.java

Open this file:

```
src/main/java/com/example/App.java
```

Replace the complete content with this code:

```
package com.example;

public class App {
    public static void main(String[] args) {
        System.out.println("Maven and Docker Integration Successful!");
    }
}
```

Save and close App.java.

Step 7: Create and Open Dockerfile

Create a new file in the root project folder, at the same level as pom.xml. The file name must be exactly:

```
Dockerfile
```

Open the Dockerfile and add this content:

```
FROM eclipse-temurin:17-jre
ARG JAR_FILE
COPY ${JAR_FILE} app.jar
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

Save and close Dockerfile.

Step 8: Package the Maven Project

Open terminal inside the project root folder and run:

```
mvn clean package
```

This command compiles the Java code and creates a JAR file inside the target folder. Expected file: target/maven-docker-demo-1.0-SNAPSHOT.jar

Step 9: Test the JAR File Locally

Before using Docker, test the JAR file locally:

```
java -jar target/maven-docker-demo-1.0-SNAPSHOT.jar
```

Expected output: Maven and Docker Integration Successful!

Step 10: Build Docker Image Using Maven Plugin

Now build the Docker image through Maven using the dockerfile-maven-plugin:

```
mvn dockerfile:build
```

This command reads the Dockerfile, copies the generated JAR file, and creates a Docker image named maven-docker-demo:latest.

Step 11: Verify Docker Image

Check whether the Docker image was created successfully:

```
docker images
```

Expected image name: maven-docker-demo latest

Step 12: Run Docker Container

Run the Docker image as a container:

```
docker run --rm maven-docker-demo:latest
```

Expected output: Maven and Docker Integration Successful!

Step 13: Complete One-Line Build and Run Flow

After the first successful setup, you can use this complete flow:

```
mvn clean package dockerfile:build
docker run --rm maven-docker-demo:latest
```

Step 14: Optional - Push Docker Image

To push the image to Docker Hub, first login:

```
docker login
```

Then update the repository name in pom.xml: <repository>your-dockerhub-username/maven-docker-demo</repository>. After that run: mvn dockerfile:push.

Step 15: Common Errors and Solutions

Error: Cannot connect to Docker daemon

Reason: Docker is not running Solution: Start Docker Desktop or Docker service.

Error: No Dockerfile found

Reason: Dockerfile is not in root folder Solution: Place Dockerfile beside pom.xml.

Error: no main manifest attribute

Reason: Main class missing in JAR manifest Solution: Use maven-jar-plugin with mainClass com.example.App.

Error: COPY failed

Reason: JAR file not created Solution: Run mvn clean package before dockerfile:build.

Error: Malformed POM

Reason: pom.xml has invalid XML Solution: Ensure root tag is <project>, not <dependency>.

Step 16: Final Student Task

Students must create a Maven Java project, configure pom.xml, create Dockerfile, build the JAR, build Docker image using dockerfile-maven-plugin, run the container, and submit screenshots of the successful Maven build, Docker image list, and container output.

Final Expected Output

```
Maven and Docker Integration Successful!
```

Submission Checklist

[ ] pom.xml completed and saved.

[ ] App.java completed and saved.

[ ] Dockerfile created in root folder.

[ ] mvn clean package executed successfully.

[ ] mvn dockerfile:build executed successfully.

[ ] docker images shows maven-docker-demo:latest.

[ ] docker run command prints the expected output.